Recurrent Neural Networks (RNNs)

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

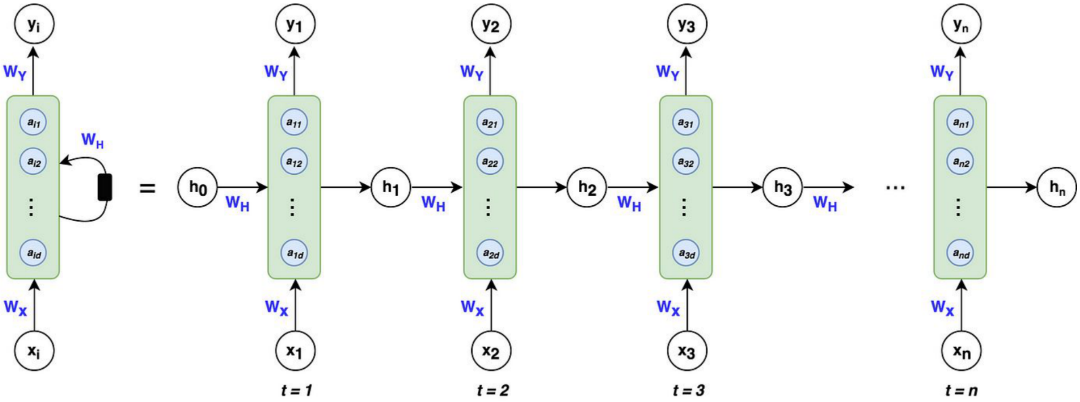
KAUST Academy

King Abdullah University of Science and Technology

July 16, 2026

Table of Contents

Motivation: Why Do We Need RNNs?



Why Do We Need RNNs?

- Traditional feedforward networks don't handle sequential data effectively.
- Many applications (language modeling, time-series prediction, speech recognition) require memory of past inputs.
- RNNs enable temporal dynamic behavior by maintaining hidden states.

Examples Where Order Matters:

- Translating "I am happy" vs "Happy I am"
- Predicting next stock price based on past trends
- Understanding a sentence word-by-word

Key Idea: Add a feedback loop to remember previous computations.

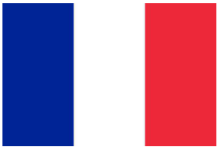
Learning Outcomes

By the end of this session, you should be able to:

- Understand the structure and working of Recurrent Neural Networks (RNNs).
- Recognize various RNN architectures (One-to-Many, Many-to-One, Many-to-Many).
- Explain the concept of shared parameters in RNNs.
- Explain why RNNs suffer from vanishing and exploding gradients.
- Understand how Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures solve these problems.
- Compare LSTM and GRU architectures in terms of performance and complexity.
- Recognize limitations and future directions in sequence modeling.

RNNs : **Introduction**

Traditional Language Models



J'ai vu le match de foot

Sequence	$P(\text{Sequence})$
I saw the game of soccer	4.5 e-5
$\boxed{\text{I saw the soccer game}}$	6.0 e-5
I saw the soccer match	4.6 e-5
Saw I the game of soccer	2.6 e-9

- Bigrams: $P(w_2|w_1) = \frac{count(w_1, w_2)}{count(w_1)}$
- Trigrams: $P(w_3|w_1, w_2) = \frac{count(w_1, w_2, w_3)}{count(w_1, w_2)}$
- $P(w_1, w_2, w_3) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2)$
- **Limitations:** Large N-grams need significant space and RAM.

N-grams: Limitations

- N-gram tables consume a lot of memory.
- **Fixed context window:** only the previous $(n-1)$ words are conditioned on.
- **Data sparsity:** larger n implies exponentially many n-grams; many combinations are rare or unseen in training data.
- **No learning or adaptability:** counts and normalization only—not learned representations in the neural sense.
- Different types of **RNNs** are the **preferred alternative**.

What is an RNN?

A neural network with loops allowing information to persist. **Core Elements:**

- Hidden State h_t captures memory of previous inputs.
- Input X_t, Output y_t.
- Parameter sharing: Same weights used across time steps.

Mathematical Formulation:

- $h_t = tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$
- $y_t = W_{hy}h_t + b_y$

This recurrence allows information to propagate through time.

Why Use RNNs?

- RNNs target **sequential data** where order matters (language, time series, speech).
- **Capturing context:** a hidden state carries information from previous steps.
- Example: word meaning often depends on preceding words; RNNs model such dependency.
- **Variable-length inputs** within a single architecture.
- **Temporal patterns** in language modeling, speech, financial series, etc.

Why Use RNNs

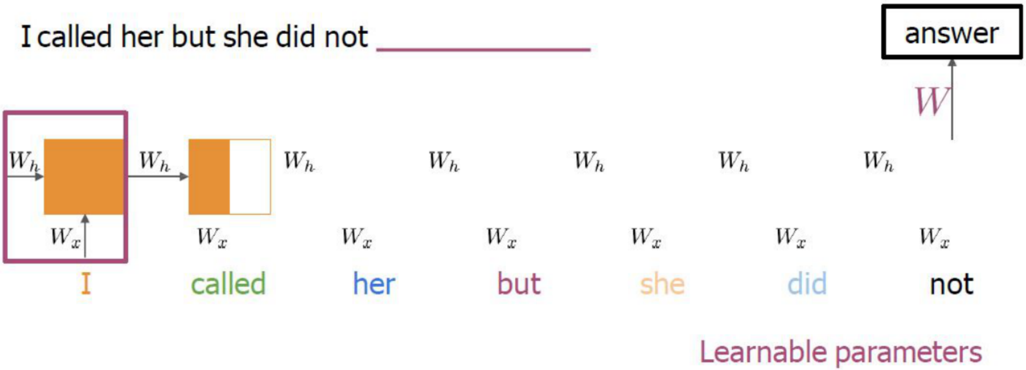
Nour was supposed to study with me. I called her but she did not answer

want

respond
choose
want
have
ask
attempt
answer
know

RNNs look at every previous word

Similar probabilities with trigram


I called her but she did not ____________
answer
W
W_h W_h W_h W_h W_h W_h W_h
W_x W_x W_x W_x W_x W_x W_x
I called her but she did not
Learnable parameters

RNN : **Architecture**

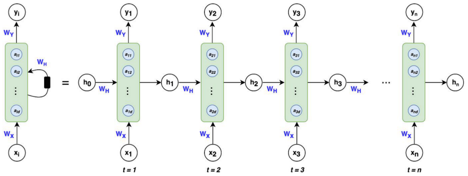
RNN Architecture: Unrolled View

Unrolled RNN:
An RNN is essentially a chain of repeating neural network modules, one for each time step.

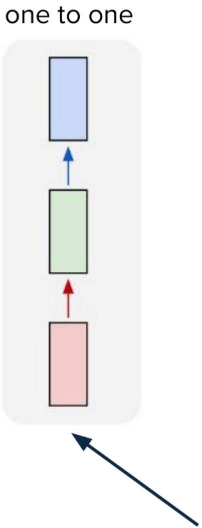
Each time step shares the same parameters:

- Input x_t
- Hidden state h_t
- Output y_t

Key Idea
Temporal representation without increasing parameter count!

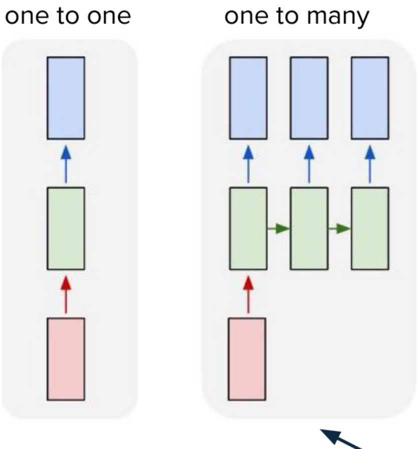


Vanilla Neural Network



Vanilla Neural Networks

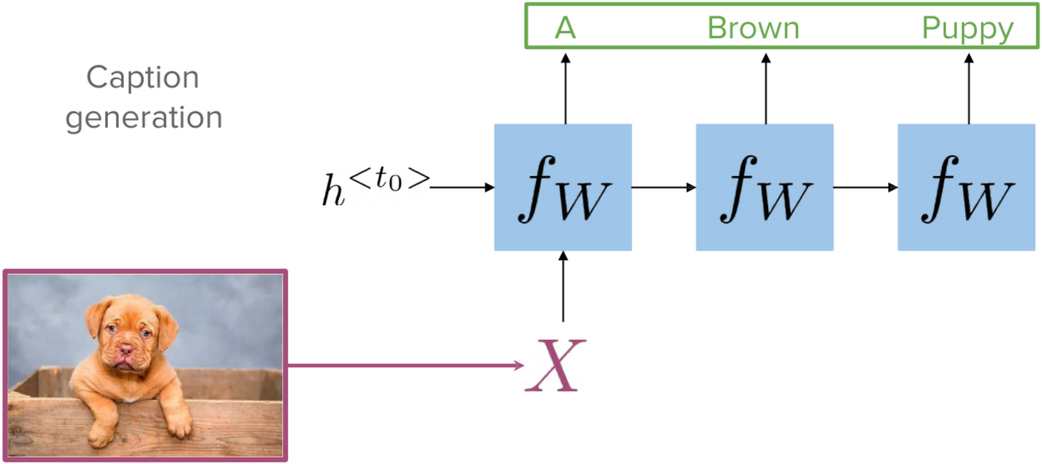
LaLiga
Santander
Real Valladolid
Real Zaragoza
Atletico Madrid
0 1
0 4
0 1
Real Madrid
Real Madrid
Real Madrid
$h^{<t_0>}$
f_W
X



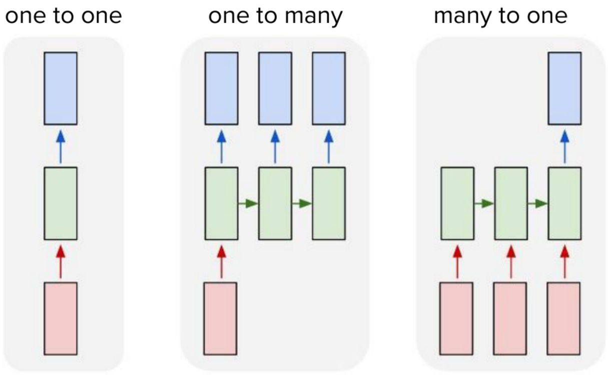
e.g. Image Captioning
image ➔ sequence of words

One to Many

Caption generation

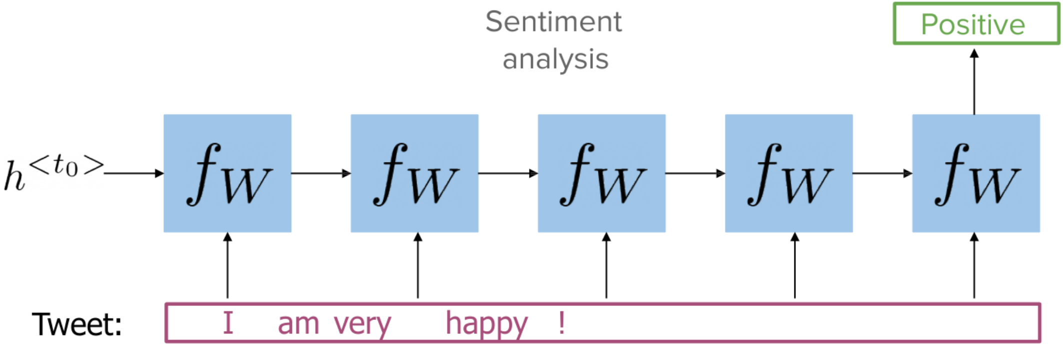


RNN: Process Sequences

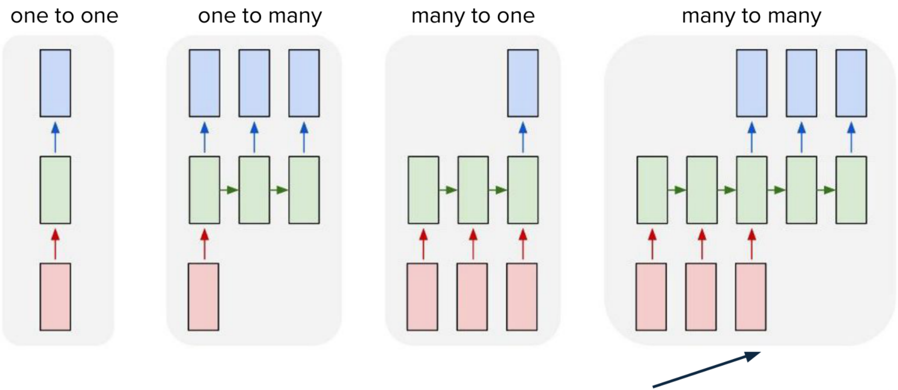


e.g. Sentiment Analysis
sequence of words ➔ sentiment

e.g. Action Prediction
sequence of video frames ➔ action class

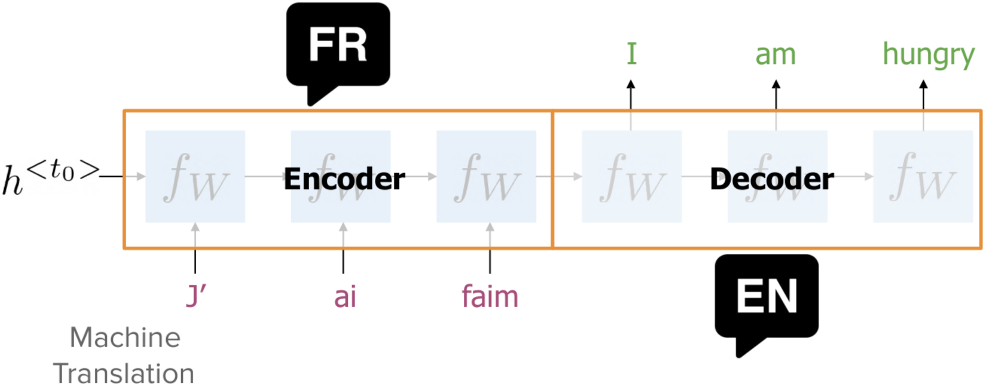

Sentiment analysis
Positive
$h^{<t_0>}$
f_W
f_W
f_W
f_W
f_W
Tweet:
I am very happy !

RNN: Process Sequences

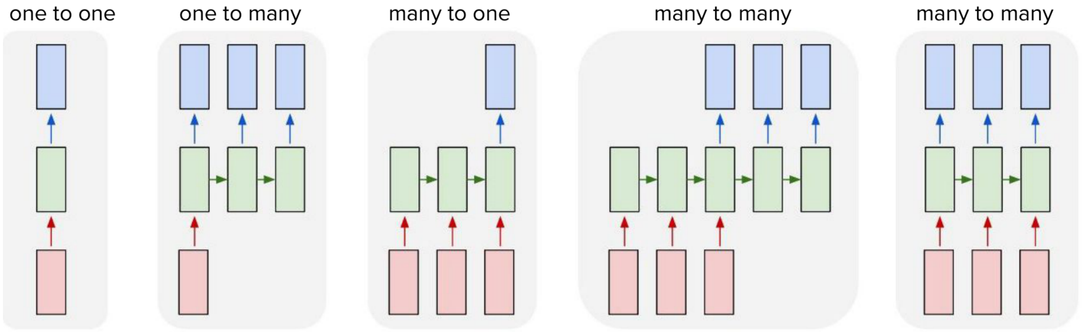


e.g. Video Captioning
sequence of video frames ➜ caption

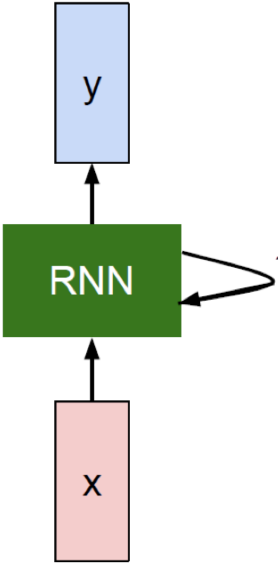
e.g. Machine Translation
sequence of words (English) ➜ sequence of words (French)


FR
I
am
hungry
$h^{<t_0>}$
Encoder
Decoder
J'
ai
faim
EN
Machine Translation

RNN: Process Sequences

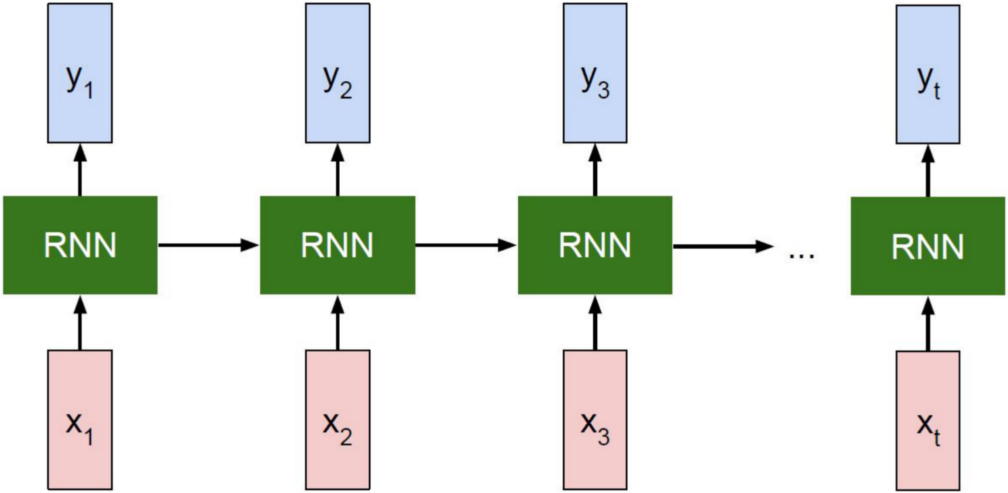


e.g. Video classification on frame level



Key idea: RNNs have an "internal state" that is updated as a sequence is processed

Unrolled RNN



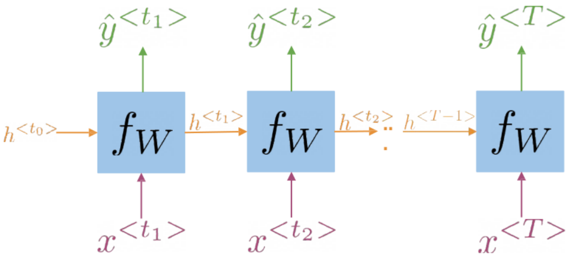
Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

RNN : **Mathematical Formulations**

Math in Simple RNNs

- How RNNs propagate information (Through time!)
- How RNNs make predictions

A Vanilla RNN

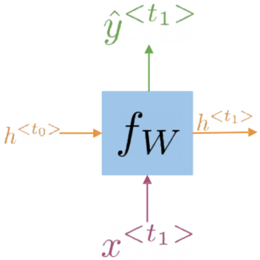


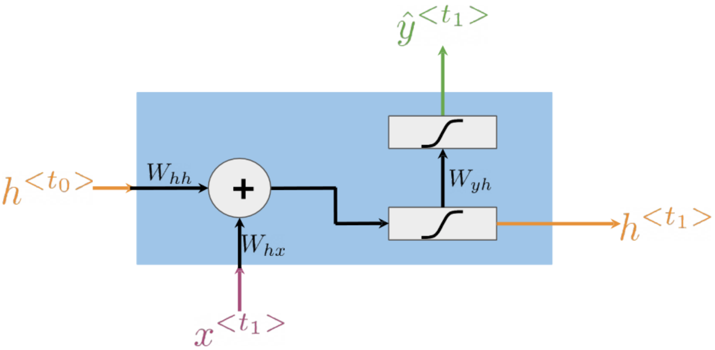
$$h^{<t>} = g(\boxed{W_h[h^{<t-1>}, x^{<t>}]}\boxed{+\, b_h})$$

$$\hat{y}^{<t>} = g(W_{yh}h^{<t>} + b_y)$$

$$\boxed{h^{<t>} = g(W_{hh}h^{<t-1>} + W_{hx}x^{<t>} + b_h)}$$

A Vanilla RNN

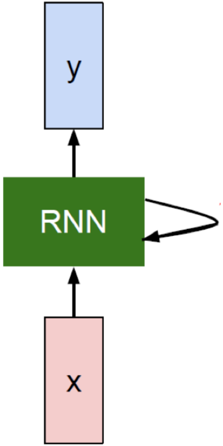




$$h^{<t>} = g(\boxed{W_{hh}h^{<t-1>} + W_{hx}x^{<t>}} + b_h)$$

$$\hat{y}^{<t>} = g(W_{yh}h^{<t>} + b_y)$$

A Vanilla RNN



The state consists of a single "hidden" vector h:

$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

$$\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t)$$

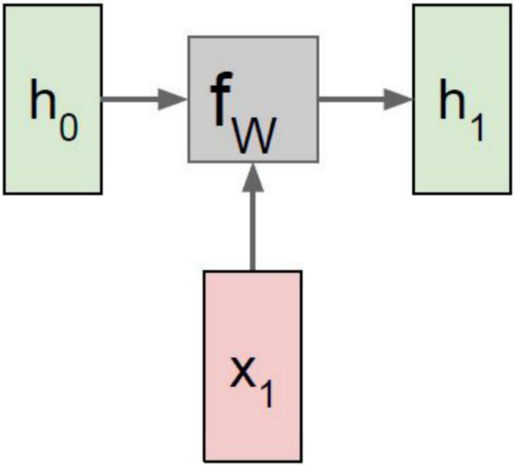
$$\mathbf{y}_t = W_{hy}\mathbf{h}_t$$

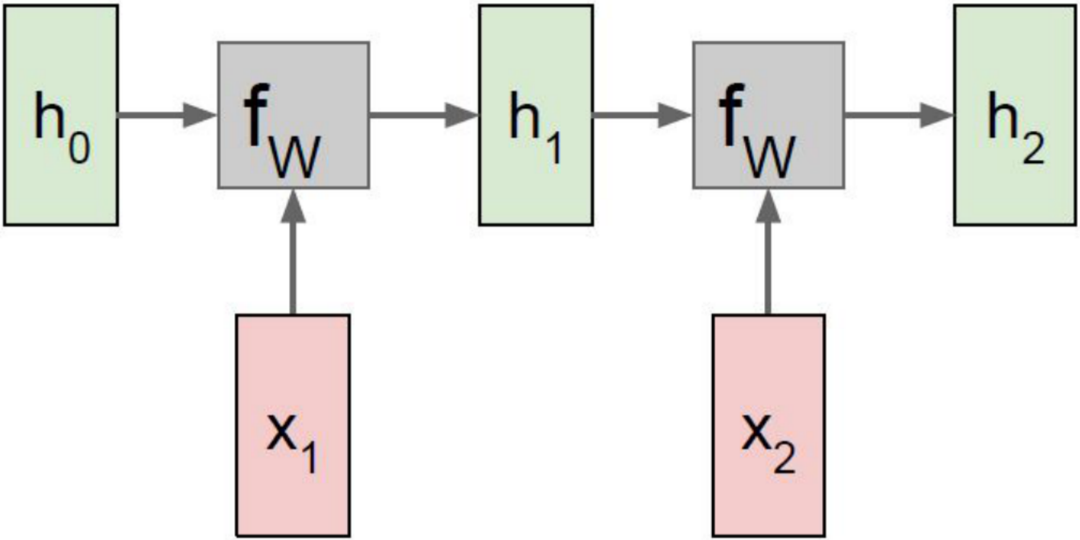
- Sometimes called a "Vanilla RNN" or an *Elman RNN* after Prof. Jeffrey Elman.

Adapted from: Stanford CS224d Lectures (R. Socher), 2023 / CSD126 (Juan Durantez, 2023)

RNN : **Parameter Sharing**

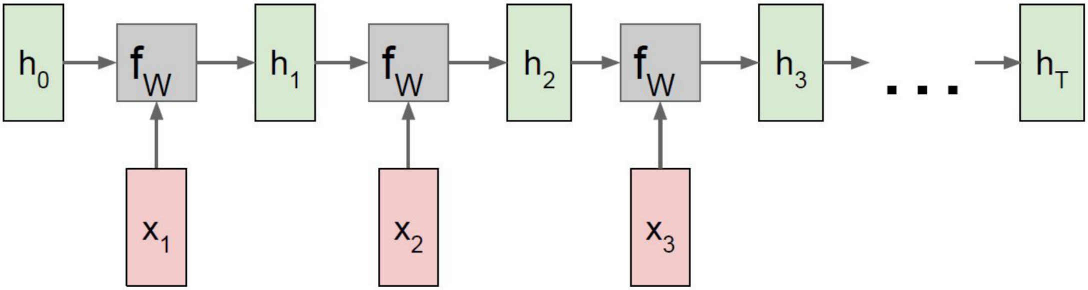
RNN: Computation Graph





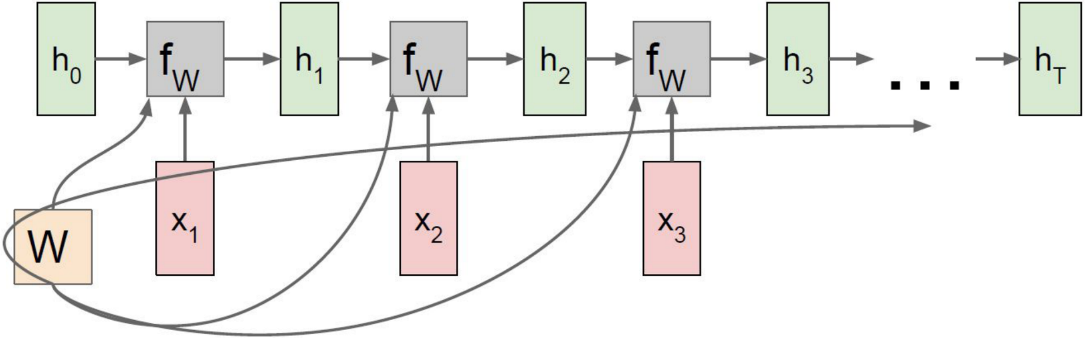
h0
fW
h1
fW
h2
x1
x2

RNN: Computation Graph

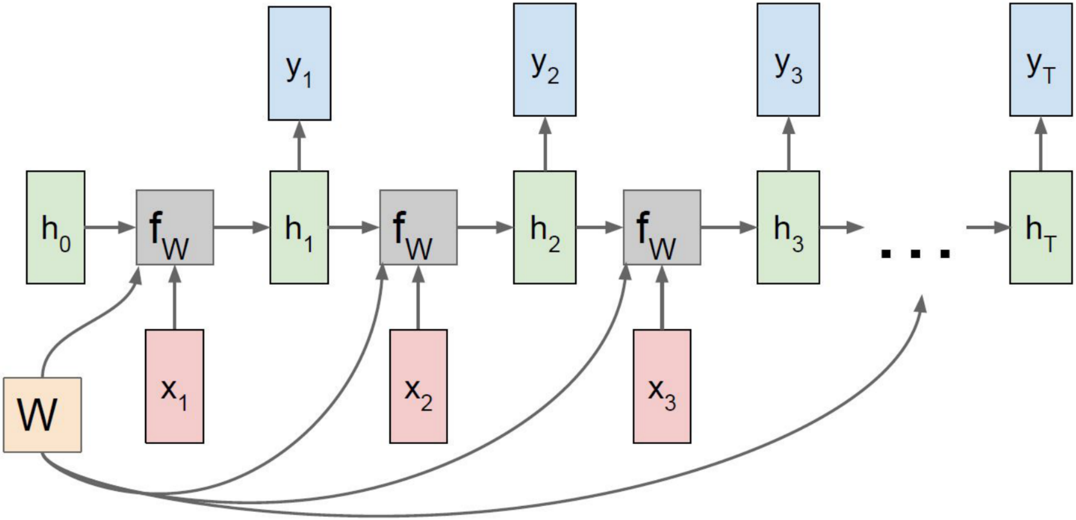


RNN: Computation Graph

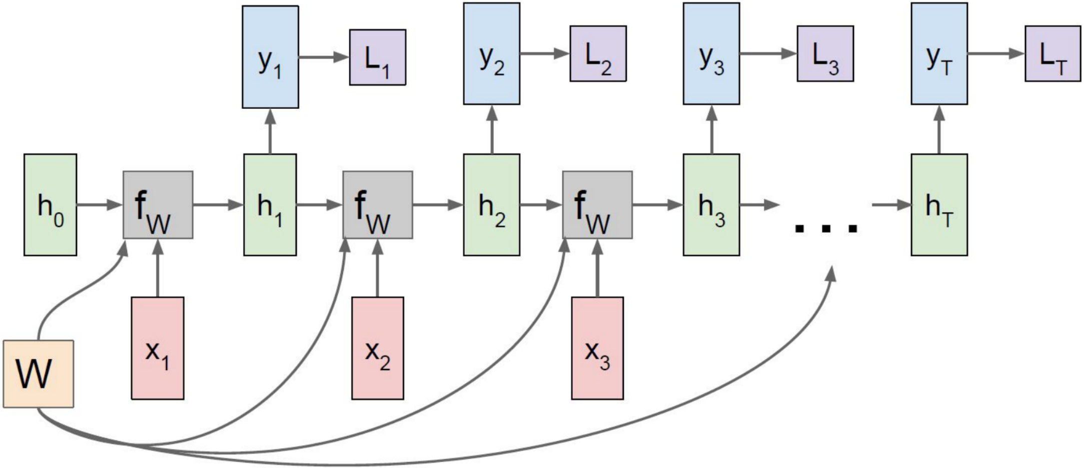
Re-use the same weight matrix at every time-step



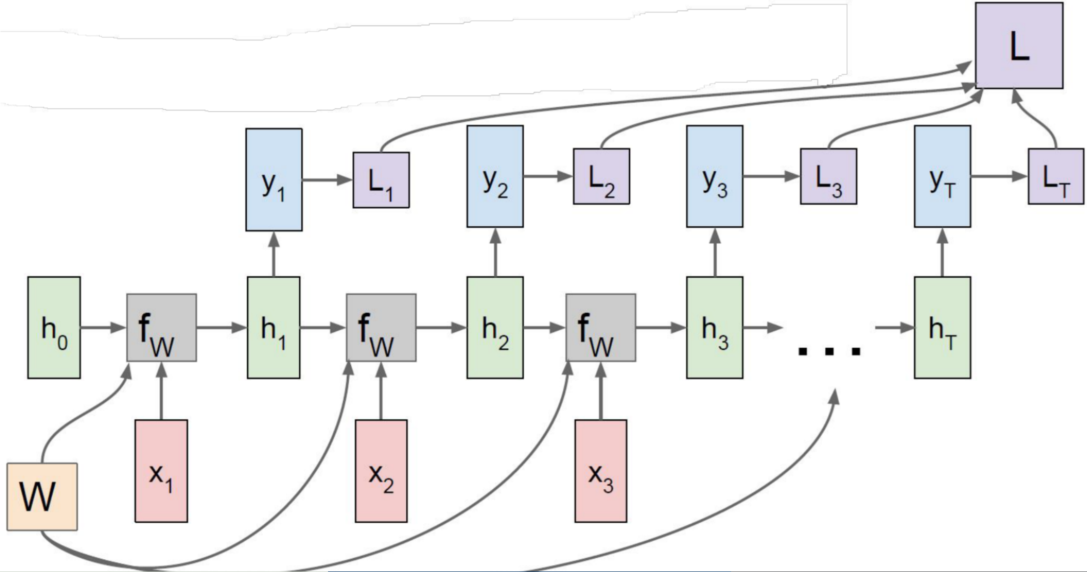
RNN Computation Graph - Many to Many



RNN Computation Graph - Many to Many

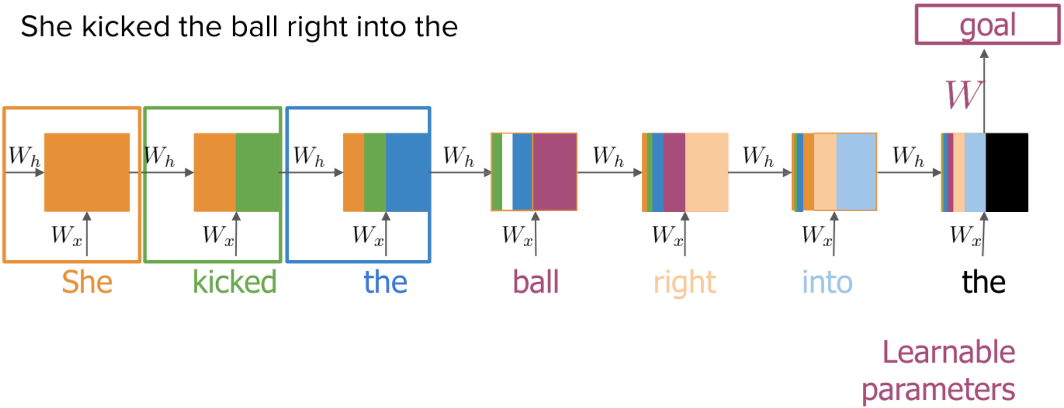


RNN Computation Graph - Many to Many



RNN : **Challenges**

RNN Basic Structure

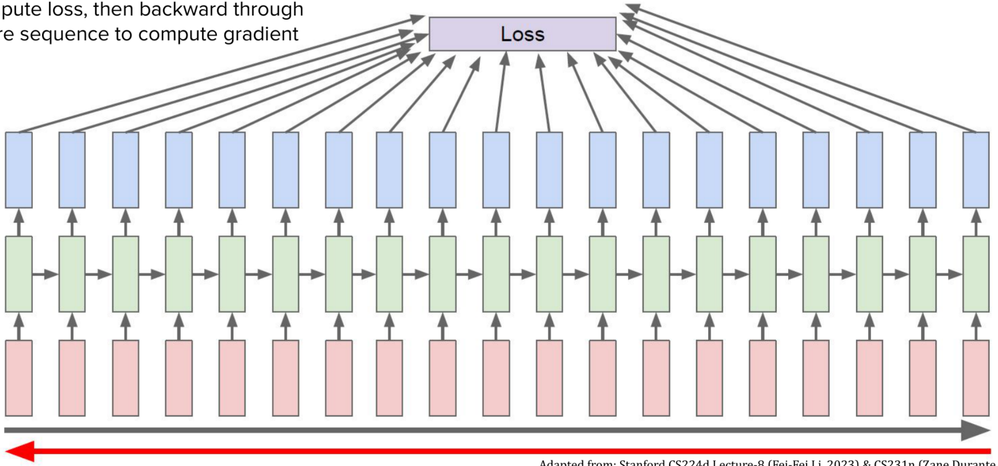


RNN Training Challenges

Backpropagation Through Time (BPTT)

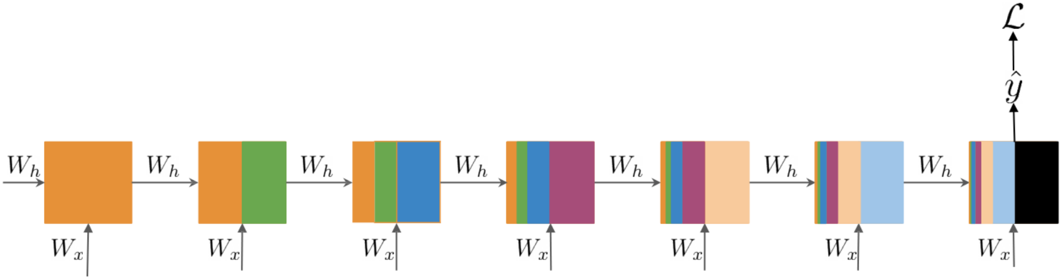
- Training RNNs involves unfolding the network across time steps.
- Standard backpropagation is applied through this unrolled structure.

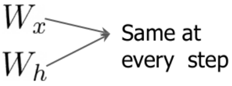
Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

Backpropagation through time





$$\frac{\partial L}{\partial W_h} \propto \sum_{1 \le k \le t} \left(\prod_{t \ge i > k} \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W_h}$$

Gradient is proportional to a sum of partial derivative products

أكاديمية كاوست
KAUST ACADEMY

$$\frac{\partial L}{\partial W_h} \propto \sum_{1\leq k\leq t} \boxed{\left(\boxed{\prod_{t\geq i>k} \frac{\partial h_i}{\partial h_{i-1}}}\right) \frac{\partial h_k}{\partial W_h}}$$

Contribution of hidden state ***k***

Length of the product proportional to how far ***k*** is from ***t***

$$\boxed{\frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{\partial h_{t-2}}{\partial h_{t-3}} \frac{\partial h_{t-3}}{\partial h_{t-4}} \frac{\partial h_{t-4}}{\partial h_{t-5}} \frac{\partial h_{t-5}}{\partial h_{t-6}} \frac{\partial h_{t-6}}{\partial h_{t-7}} \frac{\partial h_{t-7}}{\partial h_{t-8}} \frac{\partial h_{t-8}}{\partial h_{t-9}} \frac{\partial h_{t-9}}{\partial h_{t-10}} \frac{\partial h_{t-10}}{\partial W_h}}$$

Contribution of hidden state ***t-10***

$$\frac{\partial L}{\partial W_h} \propto \sum_{1\le k\le t} \boxed{\left(\boxed{\prod_{t\ge i>k} \frac{\partial h_i}{\partial h_{i-1}}} \right) \frac{\partial h_k}{\partial W_h}} \longrightarrow \text{Contribution of hidden state } k$$

Length of the product proportional to how far ***k*** is from ***t***

Partial derivatives <1	Contribution goes to 0	Vanishing Gradient
Partial derivatives >1	Contribution goes to infinity	**Exploding** Gradient

RNN Training Challenges

Problems

- **Vanishing Gradients:** Gradients shrink as they are propagated back, making it hard to learn long-term dependencies.
- **Exploding Gradients:** Gradients grow exponentially, leading to unstable updates.

Solutions Preview (covered in future modules)

- Use of LSTM and GRU architectures
- Gradient Clipping

The problem when training generative RNNs

- A generating RNN predicts token y_t from its own previous output $\hat{y}_{t-1}$.
- Early in training the model's outputs are mostly wrong, so errors feed back as inputs and compound across the sequence.

Teacher forcing

- During training, feed the ground-truth token y_{t-1} as the input at step t, regardless of what the model predicted.
- Every step is trained on a correct prefix, so learning is faster and more stable.
- At inference there is no ground truth: the model must consume its own predictions.

Exposure Bias and Scheduled Sampling

- **Exposure bias:** with teacher forcing, the model is trained only on ground-truth prefixes but tested on its own imperfect prefixes. One early mistake at inference can push it into states it never saw during training.
- **Scheduled sampling** (Bengio et al., 2015): during training, feed the model its own prediction instead of the ground truth with some probability, and increase that probability over the course of training.
- The model gradually learns to recover from its own mistakes while keeping the stability of teacher forcing early on.

Backpropagation Through Time (BPTT) spreads gradients across many time steps.

Vanishing Gradients:

$$\left\| \frac{\partial \mathcal{L}}{\partial h_t} \right\| \to 0$$

- Early layers barely learn
- Forget long-term dependencies

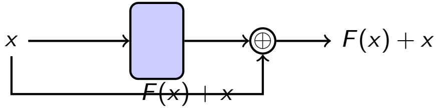
Exploding Gradients:

$$\left\| \frac{\partial \mathcal{L}}{\partial h_t} \right\| \to \infty$$

- Unstable updates, diverging weights

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

- Identity RNN with ReLU activation
- Gradient clipping
- Skip connections



RNN tradeoffs

RNN Advantages

- Can process any length input.
- Computation for step t can (in theory) use information from many steps back.
- Model size doesn't increase for longer input.
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

LSTM : **Long short term memory unit**

Introduced by Hochreiter & Schmidhuber (1997)

Core Idea: LSTM uses **gates** to control what to keep, forget, and output.

Key Components:

- **Forget Gate** f_t
- **Input Gate** i_t
- **Cell State** C_t
- **Output Gate** o_t

KAUST ACADEMY

Vanilla RNN

$$\boxed{h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)}$$

LSTM

Four gates

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state

$$c_t = f \odot c_{t-1} + i \odot g$$

Hidden state

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, “Long Short Term Memory”, Neural Computation 1997

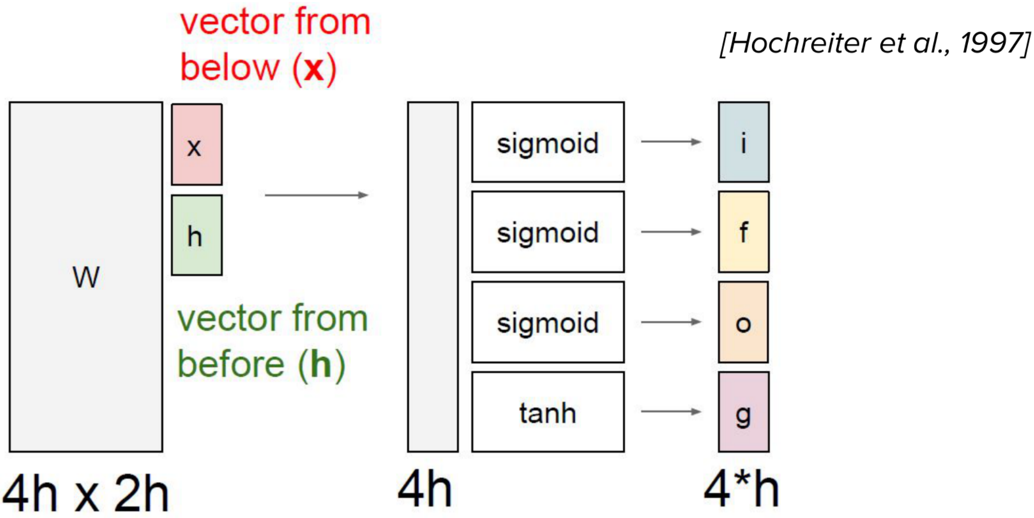
Equations:

$$
\begin{aligned}
f_t &= \sigma\left(W_f[x_t, h_{t-1}] + b_f\right) \\
i_t &= \sigma\left(W_i[x_t, h_{t-1}] + b_i\right) \\
\tilde{C}_t &= \tanh\left(W_c[x_t, h_{t-1}] + b_c\right) \\
C_t &= f_t * C_{t-1} + i_t * \tilde{C}_t \\
o_t &= \sigma\left(W_o[x_t, h_{t-1}] + b_o\right) \\
h_t &= o_t * \tanh\left(C_t\right)
\end{aligned}
$$

Helps retain long-term dependencies

LSTMs - A Memorable Solution

- Learns when to remember and when to forget
- **Basic anatomy:**
  - A cell state
  - A hidden state
  - Multiple gates
- Gates allow gradients to avoid vanishing and exploding

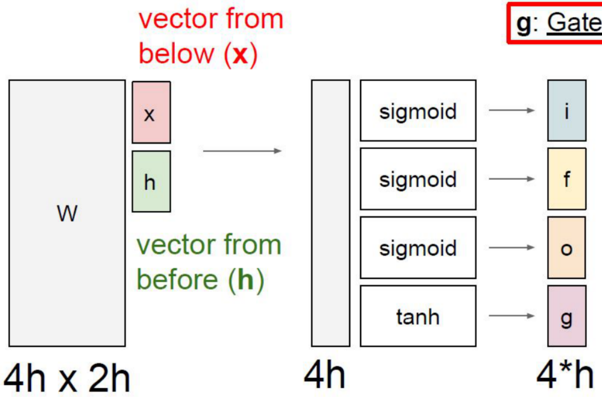


Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

Long Short-Term Memory

[Hochreiter et al., 1997]

g: Gate gate (?), How much to write to cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \boxed{\tanh} \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

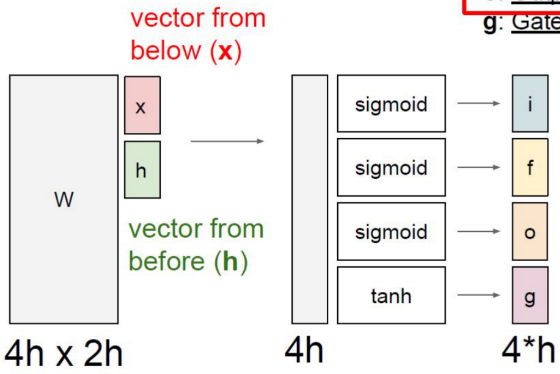
$$c_t = f \odot c_{t-1} + i \odot \boxed{g}$$

$$h_t = o \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

[Hochreiter et al., 1997]

i: Input gate, whether to write to cell
f: Forget gate, Whether to erase cell
o: Output gate, How much to reveal cell
g: Gate gate (?), How much to write to cell



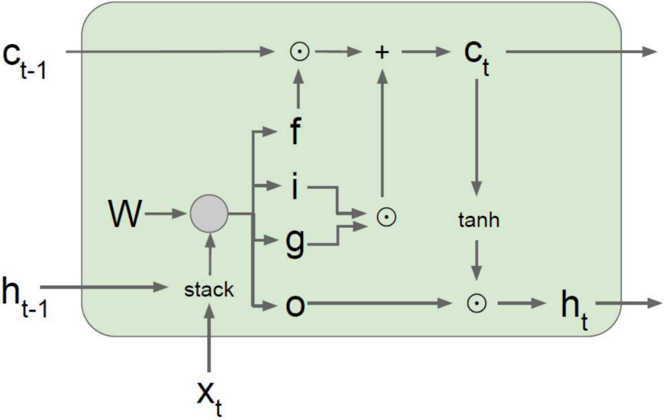
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \boxed{\sigma} \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = \boxed{o} \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

[Hochreiter et al., 1997]

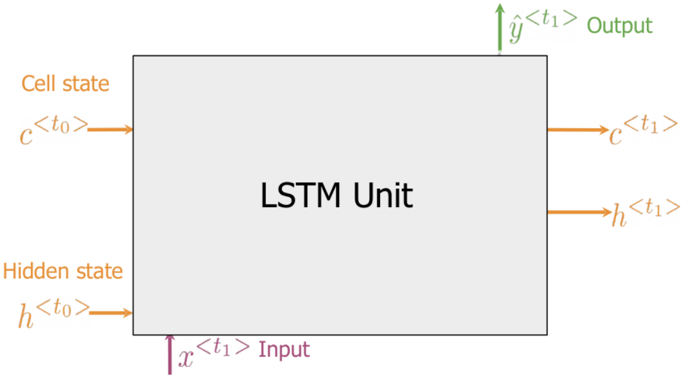


$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

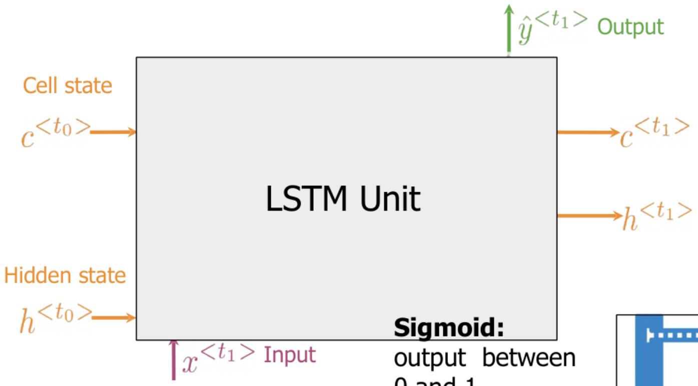
$$h_t = o \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)



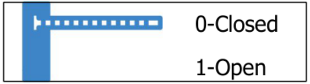
1. **Forget Gate:** information that is no longer important
2. **Input Gate:** information to be stored
3. **Output Gate:** information to use at current step

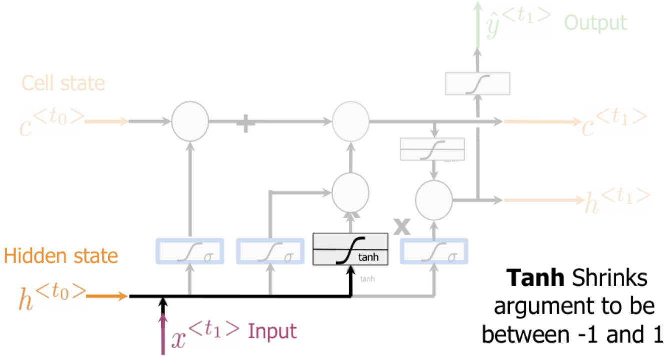
Gates in LSTM



1. **Forget Gate:** information that is no longer important
2. **Input Gate:** information to be stored
3. **Output Gate:** information to use at current step

Sigmoid: output between 0 and 1



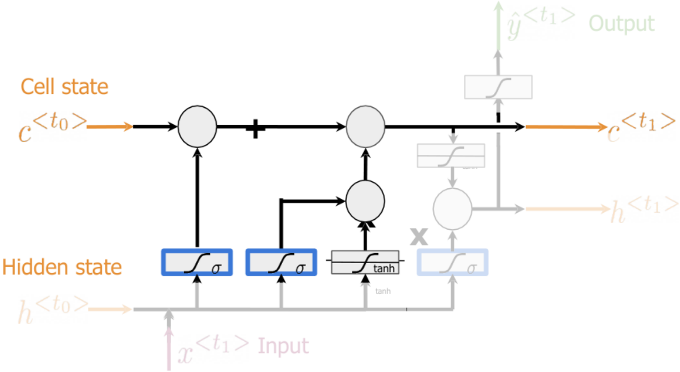


Candidate cell state

Information from the previous **hidden state** and current **input**

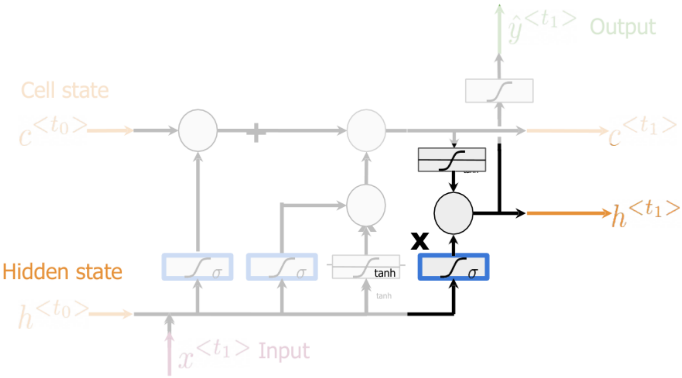
Tanh Shrinks argument to be between -1 and 1 → **Other activations could be used**

New Cell State



New Cell state

Add information from the **candidate cell state** using the **forget** and **input gates**

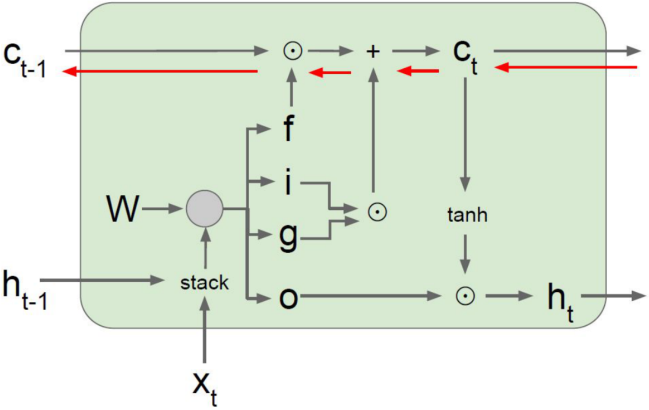


New Hidden State

Select information from the **new cell state** using the **output gate**

The **Tanh** activation could be omitted

LSTM: Gradient Flow



[Hochreiter et al., 1997]

Backpropagation from c_t to c_{t-1} only elementwise multiplication by f, no matrix multiply by W

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

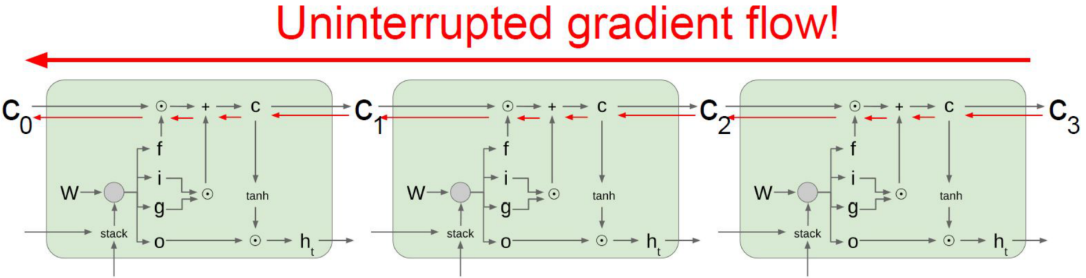
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

LSTM: Gradient Flow

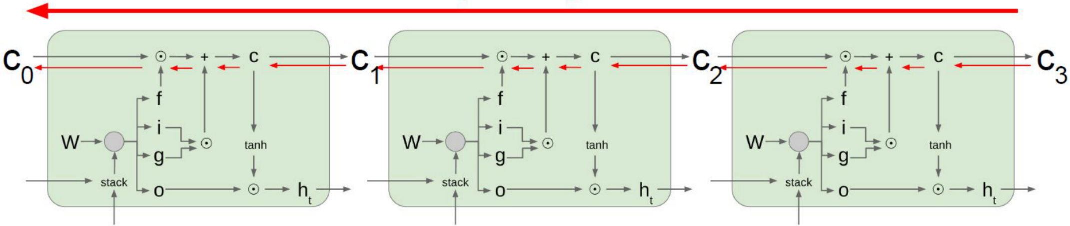
[Hochreiter et al., 1997]



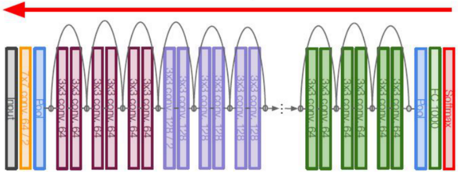
Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

[Hochreiter et al., 1997]

Uninterrupted gradient flow!



Similar to ResNet!



In between:

Highway Networks

$$g = T(x, W_T)$$

$$y = g \odot H(x, W_H) + (1 - g) \odot x$$

Srivastava et al, "Highway Networks", ICML DL Workshop 2015

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

Do LSTMs solve the vanishing gradient problem?

- The LSTM architecture makes it easier for the RNN to preserve information over many timesteps.
  - e.g. if the forget gate $f = 1$ and input gate $i = 0$, then the information of that cell is preserved indefinitely.
  - By contrast, it is harder for a vanilla RNN to learn a recurrent weight matrix W_h that preserves information in the hidden state.
- LSTM **doesn't guarantee** that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies.

GRU : **Gated Recurrent Unit**

GRU - Gated Recurrent Unit

Introduced by Cho et al., 2014

Core Idea: GRU uses **gates** to control what to keep, forget, and output.

Key Components:

- **Update Gate** z_t
- **Reset Gate** r_t

Equations:

$$
\begin{aligned}
z_t &= \sigma(W_z[x_t, h_{t-1}] + b_z) \\
r_t &= \sigma(W_r[x_t, h_{t-1}] + b_r) \\
\tilde{h}_t &= \tanh(W_h[x_t, r_t * h_{t-1}] + b_h) \\
h_t &= (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t
\end{aligned}
$$

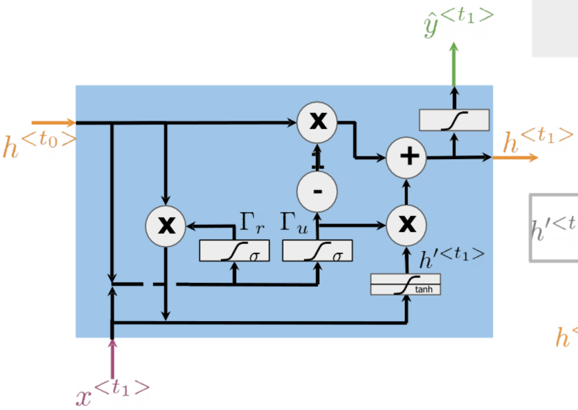
Comparable performance to LSTM
Faster training, fewer parameters

GRU - Gated Recurrent Units

"Ants are really interesting. They are everywhere."

Plural

Relevance and update gates to remember important prior information



Gates to keep/update relevant information in the hidden state

$$\Gamma_r = \sigma(W_r[h^{<t_0>}, x^{<t_1>}] + b_r)$$
$$\Gamma_u = \sigma(W_u[h^{<t_0>}, x^{<t_1>}] + b_u)$$

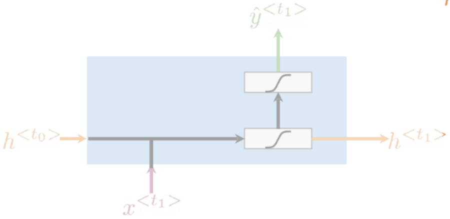
$$h'^{<t_1>} = \tanh\left(W_h[\Gamma_r * h^{<t_0>}, x^{<t_1>}] + b_h\right)$$

Hidden state candidate

$$h^{<t_1>} = (1 - \Gamma_u) * h^{<t_0>} + \Gamma_u * h'^{<t_1>}$$

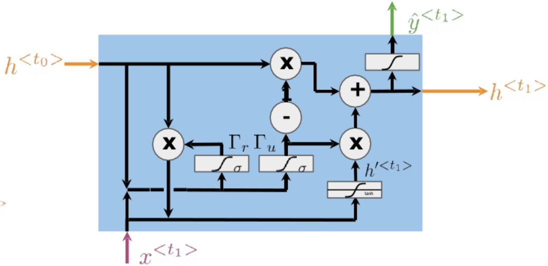
$$\hat{y}^{<t_1>} = g(W_y h^{<t_1>} + b_y)$$

Vanilla RNN vs GRUs



$$h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h)$$

$$\hat{y}^{<t>} = g(W_{yh}h^{<t>} + b_y)$$



$$\Gamma_u = \sigma(W_u[h^{<t_0>}, x^{<t_1>}] + b_u)$$

$$\Gamma_r = \sigma(W_r[h^{<t_0>}, x^{<t_1>}] + b_r)$$

$$h'^{<t_1>} = \tanh\left(W_h[\Gamma_r * h^{<t_0>}, x^{<t_1>}] + b_h\right)$$

$$h^{<t_1>} = (1 - \Gamma_u) * h^{<t_0>} + \Gamma_u * h'^{<t_1>}$$

$$\hat{y}^{<t_1>} = g(W_y h^{<t_1>} + b_y)$$

LSTM vs GRU

Feature	LSTM	GRU
Gates	3 (i, f, o)	2 (z, r)
Memory Cell	Yes	No
Complexity	Higher	Lower
Training Speed	Slower	Faster
Performance	Great for long sequences	Similar or better on short tasks

Tip: Try GRU first for faster results; switch to LSTM if performance suffers.

Use Cases for GRU/LSTM

NLP Tasks:

- Language Modeling
- Machine Translation
- Sentiment Analysis
- Chatbots

Audio & Time Series:

- Music Generation
- Speech Recognition
- Anomaly Detection

Video & Sequential Vision:

- Action Recognition
- Video Captioning

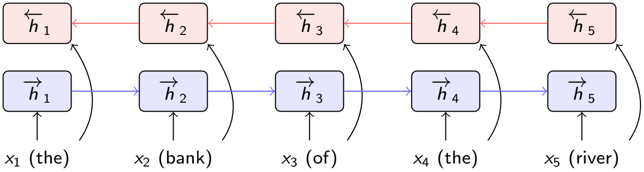
Limitations of GRU/LSTM

- Sequential computation — hard to parallelize
- Forget long-term dependencies
- Slow training due to sequential nature
- Batching variable-length sequences requires padding and masking

RNN : **Bidirectional RNNs**

Bidirectional RNNs

- A standard RNN reads left to right: h_t summarizes only the past $x_1, \ldots, x_t$.
- Many tasks need context from both sides. In "The bank of the river", the meaning of "bank" depends on words that come later.
- A **bidirectional RNN** (Schuster and Paliwal, 1997) runs two RNNs over the sequence: a forward RNN producing $\overrightarrow{h}_t$ and a backward RNN producing $\overleftarrow{h}_t$.
- The representation of each position is the concatenation $h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$: past and future context at every step.



Where Bidirectionality Works, and Where It Cannot

- **Works for encoding:** the full input is available before any output is produced.
  - Part-of-speech tagging, named entity recognition, sequence classification.
  - BiLSTMs were the standard sentence encoder before transformers.
- **Does not work for generation:** when producing text token by token, the future does not exist yet, so the backward RNN has nothing to read. Decoders stay unidirectional.
- This encoder/decoder split carries into modern models: bidirectional encoders and left-to-right decoders.

Future Directions

- **Transformers:** Fully parallelized sequence modeling using attention
- **Efficient Attention:** Longformer, Linformer, etc. for long sequences
- **Neural Memory Networks:** Explicit memory read/write
- **Recurrent Attention Models**
- **Hybrid Architectures:** RNN + CNN + Attention

RNNs are still used in edge devices for efficient modeling

Summary

- RNNs struggle with long dependencies due to vanishing/exploding gradients
- GRU and LSTM improve memory retention using gating mechanisms
- GRU is simpler and faster; LSTM is more expressive
- Attention and transformers now dominate, but RNNs remain relevant in many domains

References

These slides have been adapted from

- Younes Mourri & Lukasz Kaiser, Natural Language Processing Specialization, DeepLearning.AI

Foundational Papers:

- Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. EMNLP.
- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). *On the difficulty of training RNNs*. ICML.
- Bengio, Y., Simard, P., & Frasconi, P. (1994). *Learning long-term dependencies*. IEEE Transactions on Neural Networks.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. NeurIPS.

Resources:

- Karpathy's RNN Blog:
  https://karpathy.github.io/2015/05/21/rnn-effectiveness/
- CS231n Lecture Notes on RNNs and LSTM
- DeepLearning.ai NLP Specialization — Coursera
- MIT 6.S191 Deep Learning Lecture Slides